

Plugin Architectures with Rust

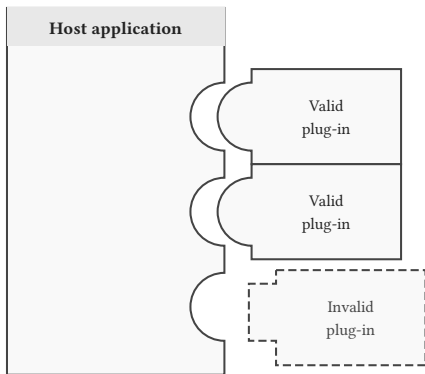
an Analysis of Obstacles
and Prospects

20.05.2026



1	Introduction	3
2	ABI	7
3	IPC	19
4	Conclusion	24

1 Introduction



“Computer Software that adds new functionality to an existing application without altering the host program itself”

Performance Overhead

Resulting from the required overhead, how slow is the execution of the plugin compared to the rust native solution.

This is measured by using a average data fusion with 1.000.000 data points and expressed with **percentages**.

Development Complexity

- Learning curve of the implementation
- Quality of the Documentation and examples
- Required amount of work and consequences for setup, configurations and integration

This is expressed from -- **to** ++.

(Language) Limitations

How much does this approach limit the usage of the rust languages features and ecosystem.

This is expressed from -- **to 0**.

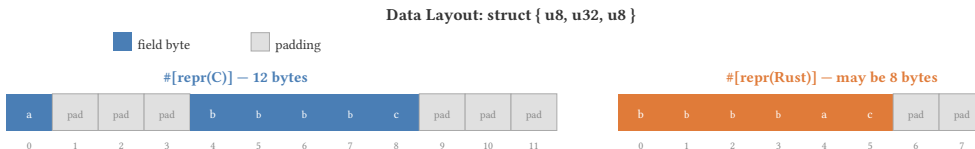
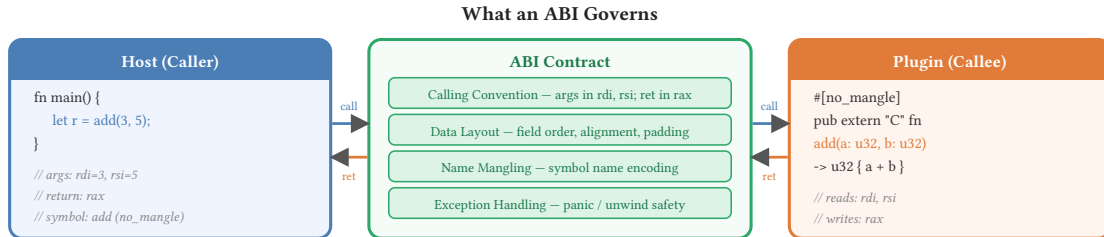
Interoperability

How well can this approach work with other libraries, tools and especially languages.

This is expressed from **0 to ++**.

2 ABI

2.a.a Application Binary Interface

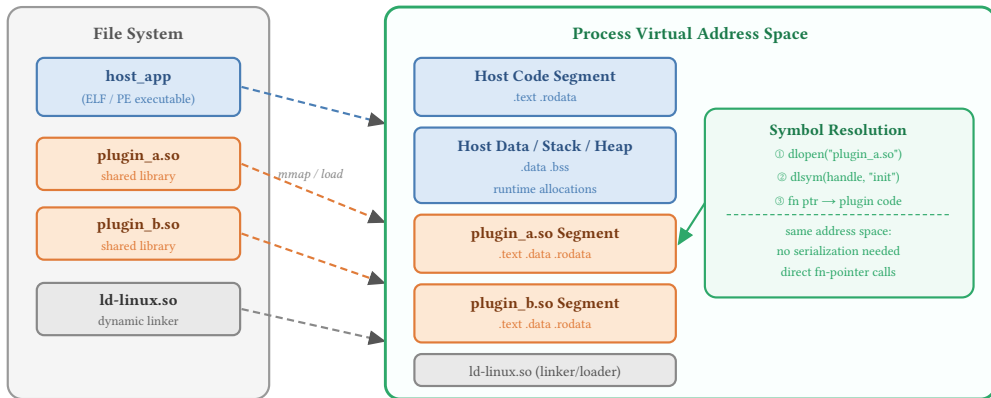


▲ The compiler may reorder fields — a C caller reading offset 8 would get garbage

Calling conventions, Data Layout, Name Mangling, Exception Handling, etc.

2.a.b Dynamic Linking/Loading

- 1) Discovery
- 2) Loading
- 3) Resolution
- 4) Execution



2.b.a Demonstration of dynamic linking and loading

```
1 // In base library (used by application and plugin):  
2 trait Fuser {  
3     fn fuse(&self, data: &[SensorData]) -> Result<SensorData, Box<dyn Error>>;  
4 }
```

 Rust

```
1 // In plugin (library):  
2 struct AveragePlugin;  
3 impl Fuser for AveragePlugin { ... }  
4  
5 #[unsafe(no_mangle)] // ensure name can be found in binary  
6 pub fn average_plugin() -> Box<dyn Fuser> {
```

 Rust

```
7     Box::new(AveragePlugin::new())  
8 }
```

```
1 // In application:
2 type FuseFunc = extern "Rust" fn() -> Box<dyn Fuser>; // use Rust ABI
3 let plugin: Plugin<FuseFunc> = Plugin::new(
4     &plugin_path.into(),
5     "average_plugin".as_bytes()
6 ).unwrap(); // 1), 2) and 3)
7
8 let fuser: Box<dyn Fuser> = plugin.get(); (1)
9 let sd = [ SensorData::new(24.0, 0.01), ... ];
10 let res = fuser.fuse(&sd); // 4) Execution
11 println!("{:?}", res);
```

 Rust

```
1 // In application:
2 struct Plugin<T> where T: Fn() -> Box<dyn Fuser> + Copy{
3     lib: Library,
4     func: T, // or Vec<T> or HashMap<String, T>
5 }
```

 Rust

Using the "libloading" crate:

- 1) Discovery
- 2) Loading
- 3) Resolution
- 4) Execution

```
1 fn new(path: &PathBuf, s: &[u8]) -> Result<Self,
   libloading::Error> {
2     let lib = unsafe { Library::new(path) }?; // 1+2)
3     let symbol: Symbol<T> = unsafe { lib.get(s) }?; // 3)
4     let func: T = *symbol; // fn deref(&self) -> &T
```

 Rust

```
5   Ok(Plugin { lib, func })  
6 }  
7 fn get(&self) -> Box<dyn Fuser> { (self.func)() }
```

2.b.b but it's Unstable

The implementation itself is not complex, but...

- Compiler version changes **may** impact the generated binary
- Changes to structs, traits and enums **may** impact the generated binary
- Base library, application and plugins need to run on the **same compiler version**

But we can use the **C ABI** (next slides)! But representing Rust in C is very difficult and will introduce trade-offs.

- How are structs, traits and enums represented?
- What happens to VTables?

- Usage of `#[sabi_trait]` attribute macro for creating FFI-safe trait objects
- Allows for a lot of customization when specifying the FFI
- Provides ABI-stable alternatives to many *std-types*, but also some *external_types*
- Widely used crate with a lot of community support

but:

- Many hidden structs and types are generated and are required for further use, therefore it is overwhelming at first
- Usage of `async` is not possible, because a FFI-safe wrapper for `Future` is not supported.

- Easy to get started as primarily `#[stabby::stabby]`, `#[stabby::export]` and `stabby::dynptr!(...)` are used.
- Good explanation of the concepts behind ABI development (and why Rust doesn't like the C ABI and vice versa)
- Provides ABI-stable alternatives to common types
- Built for performance
- “Compiler-change-proof ABI-stability is proven statically through the type system.”
- Imported/Exported binaries can be checked for validity

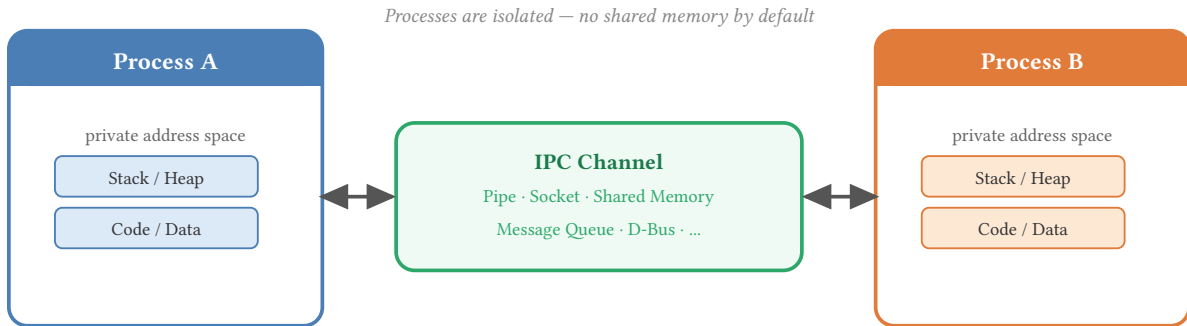
but:

- relatively new and therefore not a lot of community engagement (yet)
- had some breaking changes between versions due to a rust compiler update
- documentation can be spotty at times

Crate	Perf. Diff	Development Complexity	Limitation	Interoperability
native		++	0	0
unstable_abi	16%	--	--	0
abi_stable	25%	0	0	+
stabby	9%	0	0	+

Table 1: Summary of ABI results.

3 IPC



Biggest takeaway: Shared memory is not accessible for this approach. Therefore (de-) **serialization** is required.

High-level JSON, native Rust speed — Work with serde types, not raw pointers!

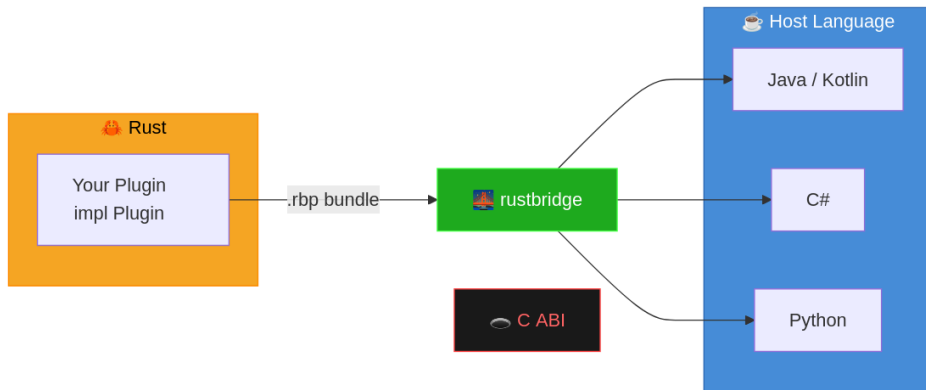


Figure 5: A rustbridge plugin can be bundled into a .rbp file and used across different languages.

- Stable C ABI — Plugins work regardless of your Rust compiler version or optimization flags
- Managed lifecycle: Startup, shutdown, including logging callbacks
- Excellent technical documentation and examples
- Communication with Request/Response Headers (JSON or Binary)
- Usage of CLI tools to create the bundle

but:

- No shared memory, therefore performance loss due to serialization
- Binary approach requires manual conversion of bytes to types
- Requires strict separation of plugin and application

Crate	Perf. Diff	Development Complexity	Limitation	Interoperability
native		++	0	0
rustbridge (JSON)	537%	+	0	++
rustbridge (Binary)	25%	-	0	++

Table 2: Summary of IPC results.

4 Conclusion

Crate	Perf. Diff	Development Complexity	Limitation	Interoperability
native		++	0	0
unstable_abi	16%	--	--	0
abi_stable	25%	0	0	+
stabby	9%	0	0	+
rustbridge (JSON)	537%	+	0	++
rustbridge (Binary)	25%	-	0	++

Table 3: Summary of results.

- Updating the plugin and host hasn't been tested
-